

TP5 – Nettoyage approfondi : Titanic

M1 Data & IA – Université Catholique de Lille

Rendu attendu

- Notebook Jupyter `.ipynb` exécutable de bout en bout.
- Fichiers de sortie : `titanic_clean.parquet`, `preprocessor.pkl`, `audit.json`.
- Travail en binôme conseillé.
- Nom du fichier : `TP5_Nettoyage_NOM_Prenom.ipynb`

Contexte

Le dataset fourni est le célèbre **Titanic** – 891 passagers du naufrage de 1912, avec leur âge, sexe, classe de billet, prix payé, port d'embarquement et survie. C'est un dataset volontairement « sale » : 20 % des âges sont manquants, 77 % des numéros de cabine inconnus, le prix du billet contient des valeurs aberrantes.

Votre mission : produire un dataset propre et un pipeline reproductible qui pourra être appliqué à de nouveaux portefeuilles.

Objectifs pédagogiques

À l'issue du TP, vous devez être capables de :

- produire un diagnostic qualité complet d'un dataset (4 dimensions) ;
- identifier le mécanisme de manque (MCAR/MAR/MNAR) et choisir la stratégie d'imputation appropriée ;
- détecter des outliers en univarié (IQR) et multivarié (Isolation Forest) ;
- dédupliquer des chaînes de caractères avec RapidFuzz ;
- construire un pipeline sklearn reproductible avec sérialisation et audit.

1. Mise en place

1.1. Installations

```
pip install pandas numpy scikit-learn seaborn matplotlib missingno rapidfuzz joblib
```

1.2. Imports

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import missingno as msno
import json, joblib, hashlib, warnings, re
from pathlib import Path
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import SimpleImputer, KNNImputer, IterativeImputer
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import Pipeline
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, RobustScaler, OneHotEncoder
from rapidfuzz import fuzz, process

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
```

1.3. Chargement du dataset

```
df = sns.load_dataset("titanic")
print(f"Dataset : {df.shape[0]} passagers, {df.shape[1]} colonnes")

# Sauvegarde d'une copie originale (à ne JAMAIS modifier)
df_brut = df.copy()
```

2. Diagnostic qualité

2.1. Profilage rapide

Question 1. Faites un premier tour d'horizon du dataset.

```
print(df.dtypes)
print()
print(df.describe(include="all").T)
print()
print("Manquants (%) :")
print((df.isna().mean() * 100).round(1).sort_values(ascending=False))
```

1. Quelles colonnes ont un type `object` (catégoriel) et lesquelles sont numériques ?
2. Quelles trois colonnes contiennent le plus de valeurs manquantes ? Donnez leur taux.
3. Y a-t-il une colonne dont l'écart-type est suspect (très grand ou nul) ? Que cela peut-il signifier ?

2.2. Visualisation des manquants

Question 2. Visualisez les patterns de manquants.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
msno.matrix(df, ax=axes[0])
msno.heatmap(df, ax=axes[1])
plt.tight_layout()
plt.show()
```

Sur la **heatmap**, deux colonnes manquent-elles ensemble plus souvent qu'au hasard ? Si oui lesquelles, et qu'est-ce que cela suggère sur le mécanisme de manque ?

2.3. Les 4 dimensions de qualité

Question 3. Calculez quatre indicateurs de qualité sur le dataset.

```
qualite = {
    "completude_globale": 1 - df.isna().mean().mean(),
    "lignes_completes_pct": (~df.isna().any(axis=1)).mean(),
    "doublons_exacts": df.duplicated().sum(),
```

```
"doublons_sur_id": df.duplicated(subset=["sex", "age", "fare", "pclass"]).sum(),
}
for k, v in qualite.items():
    print(f" {k:25s}: {v}")
```

1. Quel pourcentage de lignes est complètement renseigné ?
2. Y a-t-il des doublons exacts ? Et sur la combinaison (`sex`, `age`, `fare`, `pclass`) ?
3. Si oui, sont-ce vraiment des doublons (même personne enregistrée deux fois) ou simplement des passagers aux profils similaires ?

3. Valeurs manquantes

3.1. Identifier le mécanisme de manque

Question 4. La colonne `age` est manquante pour 20 % des passagers. Le manque est-il aléatoire ou lié à la classe sociale ?

```
# Taux de manquants sur age par classe
print(df.groupby("pclass")["age"].apply(lambda x: x.isna().mean() * 100).round(1))
print()

# Test plus fin : par classe et par sexe
print(df.groupby(["pclass", "sex"])["age"].apply(lambda x: x.isna().mean() * 100).round(1))
```

1. Le taux de manquants est-il homogène entre les classes ?
2. Quel mécanisme cela suggère-t-il : MCAR, MAR ou MNAR ? Justifiez.
3. Quelle conséquence cela a-t-il sur la stratégie d'imputation à choisir ?

3.2. Stratégies d'imputation

Question 5. Comparez trois stratégies d'imputation pour `age`.

```
# Strategie A : mediane globale
df_a = df.copy()
df_a["age"] = df_a["age"].fillna(df_a["age"].median())

# Strategie B : mediane par groupe (classe x sexe)
df_b = df.copy()
df_b["age"] = df_b.groupby(["pclass", "sex"])["age"].transform(
    lambda x: x.fillna(x.median())
)

# Strategie C : KNN (utilise plusieurs colonnes pour predire)
df_c = df.copy()
cols_num = ["age", "fare", "sibsp", "parch", "pclass"]
knn = KNNImputer(n_neighbors=5)
df_c[cols_num] = knn.fit_transform(df_c[cols_num])

# Comparer les distributions resultantes
for nom, dfx in [("A_median_globale", df_a), ("B_median_groupe", df_b), ("C_KNN", df_c)]:
    s = dfx["age"]
    print(f"{nom:20s} mean={s.mean():.1f} std={s.std():.1f} median={s.median():.1f}")
```

```
print(f"\nOriginal (avec NaN) mean={df['age'].mean():.1f} std={df['age'].std():.1f}
      median={df['age'].median():.1f}")
```

1. Quelle stratégie préserve le mieux l'écart-type original ? Pourquoi est-ce un bon critère ?
2. Quelle stratégie produit la moyenne la plus différente de l'originale ? Cela signifie-t-il qu'elle est mauvaise ?
3. Pour ce dataset, quelle stratégie recommanderiez-vous ? Justifiez en 2-3 phrases.

3.3. Cas deck – 77% de manquants

Question 6. La colonne `deck` (pont du navire) est manquante pour 77 % des passagers.

```
print("Manquants deck par classe :")
print(df.groupby("pclass")["deck"].apply(lambda x: x.isna().mean() * 100).round(1))
```

1. Le manque est-il lié à la classe ?
2. Pourquoi historiquement `deck` est mal renseigné pour les passagers de 3^e classe ?
3. Vaut-il mieux imputer ou supprimer cette colonne ? Justifiez.

4. Outliers

4.1. Détection univariée – IQR

Question 7. Détectez les outliers de `fare` (prix du billet).

```
def outliers_iqr(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - k * iqr) | (s > q3 + k * iqr)

mask = outliers_iqr(df["fare"])
print(f"Outliers fare (k=1.5) : {mask.sum()} ({mask.mean()*100:.1f} %)")
print(f"Max fare : {df['fare'].max():.0f} GBP")
print(f"Q3 + 1.5 IQR : {df['fare'].quantile(0.75) + 1.5 * (df['fare'].quantile(0.75) - df
    ['fare'].quantile(0.25)):.0f} GBP")

# Boxplot
df.boxplot(column="fare", by="pclass", figsize=(8, 4))
plt.suptitle("")
plt.title("Fare par classe")
plt.show()
```

1. Combien de passagers ont un `fare` considéré comme outlier ?
2. Sur le boxplot par classe, où se concentrent les outliers ? Est-ce surprenant ?
3. Faut-il les supprimer, les caper ou les conserver ? Justifiez en pensant au métier (assurance maritime).

4.2. IQR par groupe

Question 8. La détection IQR globale signale beaucoup de prix de 1^{re} classe comme outliers, alors qu'ils sont normaux pour leur catégorie. Refaites la détection *par classe*.

```
df["out_fare_par_classe"] = df.groupby("pclass")["fare"].transform(outliers_iqr)

print("Outliers fare par classe :")
print(df.groupby("pclass")["out_fare_par_classe"].sum())
print(f"Total : {df['out_fare_par_classe'].sum()}")
```

Combien d'outliers la détection par groupe trouve-t-elle, comparé à la détection globale ? Quelle approche est la plus juste ?

4.3. Détection multivariée – Isolation Forest

Question 9. Certains profils de passagers sont anormaux quand on regarde plusieurs variables ensemble (ex : enfant voyageant seul en 1^{re} classe).

```
cols = ["age", "fare", "sibsp", "parch", "pclass"]
X = df[cols].dropna()

iso = IsolationForest(contamination=0.05, random_state=42, n_estimators=100)
preds = iso.fit_predict(X)

X = X.copy()
X["anomalie"] = preds == -1
X["score"] = iso.decision_function(X.drop(columns=["anomalie"]))

print(f"Anomalies : {X['anomalie'].sum()} ({X['anomalie'].mean()*100:.1f} %)")
print("\n10 profils les plus anormaux :")
print(X.nsmallest(10, "score"))
```

1. Décrivez 2 profils anormaux trouvés (âge, classe, prix, famille). Sont-ils plausibles ?
2. Que signifie le score de `decision_function` (positif vs négatif) ?
3. Quel paramètre `contamination` a-t-on choisi ? Que se passe-t-il si on met 0.20 ?

5. Doublons et déduplication floue

5.1. Doublons exacts

Question 10. Y a-t-il des passagers en double ?

```
print(f"Doublons exacts : {df.duplicated().sum()}")
print(f"Sur (sex, age, fare, pclass, embarked) : {df.duplicated(subset=['sex', 'age', 'fare', 'pclass', 'embarked']).sum()}")

# Inspecter les "doublons" sur identifiant minimal
mask = df.duplicated(subset=["sex", "age", "fare", "pclass", "embarked"], keep=False)
print(f"\nLignes concernées :")
print(df[mask].sort_values(["sex", "age", "fare", "pclass"]).head(10))
```

Les lignes apparemment dupliquées sont-elles vraiment la même personne, ou des passagers distincts aux profils similaires (jumeaux, frères et sœurs voyageant ensemble) ? Justifiez.

5.2. Doublons flous sur le port d'embarquement

Question 11. Pour cet exercice, simulons une saisie manuelle imparfaite du port d'embarquement.

```
# Simulation d'une saisie sale (typos, casse, abreviations)
np.random.seed(42)
saisies_sales = np.random.choice(
    ["Southampton", "southampton", "SOUTHAMPTON", "S. Hampton",
     "Cherbourg", "cherbourg", "Cherb.",
     "Queenstown", "Queens town", "QSTOWN"],
    size=200
)
df_sale = pd.DataFrame({"port": saisies_sales})
print(f"Valeurs uniques : {df_sale['port'].nunique()}")
print(df_sale["port"].value_counts())

# Liste de reference (3 ports officiels)
ports_officiels = ["Southampton", "Cherbourg", "Queenstown"]

def resoudre(nom, ref, seuil=70):
    if pd.isna(nom):
        return None
    match = process.extractOne(str(nom), ref, scorer=fuzz.ratio)
    return match[0] if match and match[1] >= seuil else None

df_sale["port_canon"] = df_sale["port"].apply(lambda x: resoudre(x, ports_officiels))
print("\nAprès deduplication :")
print(df_sale["port_canon"].value_counts(dropna=False))
```

1. Combien de variantes orthographiques RapidFuzz a-t-il consolidées ?
2. Y a-t-il des variantes qui n'ont pas été résolues (None) ? Pourquoi ?
3. Que se passerait-il si on baisse le seuil à 50 ? Testez et commentez.

6. Transformations

6.1. Encodage des catégorielles

Question 12. La colonne `pclass` est numérique (1, 2, 3) mais représente une catégorie ordinale. `sex` et `embarked` sont nominales.

```
# pclass est déjà ordonnée (1 < 2 < 3) -> on peut la laisser numérique
# sex et embarked -> One-Hot

df_enc = pd.get_dummies(df, columns=["sex", "embarked"], drop_first=True)
print(f"Avant : {df.shape}, après OneHot : {df_enc.shape}")
print(f"Nouvelles colonnes : {[c for c in df_enc.columns if c.startswith(('sex_', 'embarked_'))]}")
```

1. Pourquoi utilise-t-on `drop_first=True` ? Quel problème statistique cela évite-t-il ?
2. `pclass` est numérique 1/2/3. Si on l'encodait en One-Hot à la place, qu'est-ce qu'on gagnerait ou perdrait ?

6.2. Transformation du fare

Question 13. La distribution de `fare` est très asymétrique. Comparez la version brute et la version log.

```
df["log_fare"] = np.log1p(df["fare"])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df["fare"].hist(bins=50, ax=axes[0])
axes[0].set_title("fare (brut)")
df["log_fare"].hist(bins=50, ax=axes[1])
axes[1].set_title("log(1 + fare)")
plt.show()
```

Quelle transformation rend la distribution la plus proche d'une normale ? Pourquoi est-ce utile pour un modèle de régression ?

7. Pipeline reproductible

7.1. Construction du pipeline

Question 14. Assemblez tout le nettoyage dans un Pipeline sklearn.

```
cols_num = ["age", "fare", "sibsp", "parch"]
cols_cat = ["sex", "embarked", "pclass"]

preproc_num = Pipeline([
    ("imputer", KNNImputer(n_neighbors=5)),
    ("scaler", RobustScaler()),
])

preproc_cat = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])

preprocesseur = ColumnTransformer([
    ("num", preproc_num, cols_num),
    ("cat", preproc_cat, cols_cat),
])

# Application sur tout le dataset (en pratique : split train/test d'abord)
X = df.drop(columns=["survived", "alive", "deck", "alone", "class", "who", "adult_male",
    "embark_town"], errors="ignore")
y = df["survived"]

X_clean = preprocesseur.fit_transform(X)
print(f"Avant : {X.shape}, apres pipeline : {X_clean.shape}")
```

1. Pourquoi met-on l'imputeur **avant** le scaler dans la séquence ?
2. Pourquoi sépare-t-on les transformations numériques et catégorielles via `ColumnTransformer` ?
3. Sur des données réelles, faudrait-il faire `fit_transform` sur tout `X` ou seulement sur `X_train` ? Justifiez.

7.2. Sérialisation et audit

Question 15. Sauvegardez le pipeline et les métadonnées du nettoyage.

```

# 1. Serialiser le pipeline (reutilisable sur de nouvelles donnees)
joblib.dump(preprocesseur, "preprocessor.pkl")

# 2. Audit : metadata du nettoyage
audit = {
    "lignes_initiales": len(df_brut),
    "lignes_finales": X_clean.shape[0],
    "colonnes_finales": X_clean.shape[1],
    "manquants_avant": df_brut.isna().sum().to_dict(),
    "imputer_strategy": "KNN k=5 sur numeriques, mode sur categorielles",
    "scaler": "RobustScaler",
    "hash_brut": hashlib.md5(
        pd.util.hash_pandas_object(df_brut).values
    ).hexdigest(),
}
with open("audit.json", "w") as f:
    json.dump(audit, f, indent=2, default=str)

# 3. Sauvegarder le dataset nettoye
df_propre = df.copy()
df_propre["age"] = df.groupby(["pclass", "sex"])["age"].transform(
    lambda x: x.fillna(x.median())
)
df_propre["embarked"] = df_propre["embarked"].fillna(df_propre["embarked"].mode()[0])
df_propre = df_propre.drop(columns=["deck", "alive", "alone", "class", "who", "adult_male",
    "embark_town"], errors="ignore")
df_propre.to_parquet("titanic_clean.parquet", index=False)

print("Sauvegardes : preprocessor.pkl, audit.json, titanic_clean.parquet")

```

Pourquoi est-ce important de :

1. Sauvegarder le pipeline avec joblib plutôt que de réécrire le code à chaque fois ?
2. Calculer un hash du dataset brut ? Donnez un cas concret où cela vous sauve.
3. Conserver une copie du dataset original (`df_brut`) jamais modifié ?

7.3. Test du pipeline sur de nouvelles données

Question 16. Vérifiez que le pipeline sauvegardé peut traiter de nouvelles lignes.

```

# Simuler de "nouvelles donnees" (par exemple un nouveau client)
nouveau = pd.DataFrame([
    {
        "age": 35, "fare": 50, "sibsp": 0, "parch": 1,
        "sex": "female", "embarked": "S", "pclass": 2,
    }
])

# Recharger le pipeline et l'appliquer
p = joblib.load("preprocessor.pkl")
nouveau_clean = p.transform(nouveau)
print(f"Nouveau passager transforme : {nouveau_clean.shape}")
print(nouveau_clean)

```

Si on changeait la liste des ports possibles dans `embarked` (par exemple ajouter un port « B » jamais vu), que ferait le pipeline ? Quel paramètre de `OneHotEncoder` gère ce cas ?

8. Synthèse

Question 17. Rédigez un rapport de 200 mots à votre client (la compagnie d'assurance), structuré ainsi :

1. **Qualité initiale du portefeuille** : quels sont les principaux problèmes identifiés ?
2. **Décisions de nettoyage** : quelles données avez-vous corrigées, supprimées, conservées telles quelles ?
3. **Limites** : quels biais subsistent dans le dataset nettoyé ? Que recommandez-vous pour les futures collectes ?

Le rapport doit être compréhensible par un actuairé qui ne code pas.

Conseils de présentation

- Structurez le notebook avec les titres des sections.
- Pour chaque question, rédigez une cellule Markdown de réponse sous le code.
- Incluez les fichiers de sortie dans le rendu.
- Vérifiez que le notebook s'exécute de bout en bout sans erreur.